

Evolving Navigation Policies for FrozenLake-v1 Using Genetic Algorithms: An Empirical Study of Operator Selection, Parameter Sensitivity, and Fitness Shaping

Arda Çağın Keser
Computer Engineering Department
Ege University
İzmir, Turkey
05240000811@ogrenci.ege.edu.tr

Osman Oğuz Erol
Computer Engineering Department
Ege University
İzmir, Turkey
05240000814@ogrenci.ege.edu.tr

Utkan Keser
Computer Engineering Department
Ege University
İzmir, Turkey
05240000826@ogrenci.ege.edu.tr

Shahram Saraeipour
Computer Engineering Department
Ege University
İzmir, Turkey
05220001308@ogrenci.ege.edu.tr

Abstract—Sparse-reward navigation tasks challenge conventional reinforcement learning because the agent receives informative feedback only at terminal states, making per-step credit assignment difficult. Direct policy search via genetic algorithms (GAs) bypasses this difficulty by encoding a complete state-to-action mapping as an evolvable genotype and evaluating it through episode-level fitness. This paper presents an empirical study of GA-based direct policy search on the deterministic 8×8 FrozenLake-v1 benchmark. Each individual encodes a tabular policy of 64 discrete actions; a shaped fitness function combines a goal-success reward, a Manhattan-distance potential, a per-step cost, and a hole-fall penalty to provide informative gradients across the sparse landscape. Three research questions are investigated: which combination of crossover operator, mutation operator, and replacement strategy achieves the best performance; how sensitive GA performance is to operator rates, population size, and tournament size; and how fitness weight choices alter evolved behavior. Results show that five of thirty tested operator combinations consistently achieve the theoretical maximum fitness of 108.60 with 100% success rate and zero run-to-run variance. Rate sensitivity analysis reveals that a mutation rate of 0.01 is critically dependent on crossover rate, while a rate of 0.2 achieves peak fitness regardless of crossover configuration. An aggressive step penalty of 2.0 per step collapses performance to 0% success, demonstrating that shaping balance is as critical as operator design. A scalability study reveals qualitatively distinct convergence regimes: the 8×8 grid reaches a high-fitness plateau by generation 20, the 16×16 grid converges much later via step-like jumps and reaches a comparable plateau only after generation 100, while the 32×32 grid fails to improve meaningfully within the same generation budget.

Index Terms—genetic algorithm, direct policy search, FrozenLake, reinforcement learning, fitness shaping, operator selection, parameter sensitivity, fitness landscape

I. INTRODUCTION

Reinforcement learning (RL) provides a principled framework for sequential decision-making, but sparse reward settings—where the reward signal is issued only at terminal states—introduce a severe credit-assignment problem. Value-function-based methods struggle to propagate meaningful learning signals backward through long trajectories when feedback is so infrequent [1]. Evolutionary computation (EC) offers a complementary approach: by encoding a complete policy as a chromosome and evaluating it through full-episode fitness, the selection signal naturally reflects trajectory-level performance without requiring per-step gradient information.

FrozenLake-v1 is a canonical gridworld benchmark in which an agent must navigate an $N \times N$ frozen surface from a start cell to a goal cell while avoiding holes. In the deterministic (non-slippery) variant, the transition function is fully specified, making it a tractable but non-trivial benchmark for policy search: the policy space is large (4^{N^2} possible deterministic policies), rewards are sparse (0 at every step, 1 only upon reaching the goal), and outcomes are binary. Its simplicity makes it useful for controlled empirical study, while its sparsity motivates the use of fitness shaping.

This paper applies GA direct policy search to the deterministic 8×8 FrozenLake-v1 environment. Each individual encodes a tabular policy as a 64-gene integer chromosome, with genes representing discrete actions {Left, Down, Right, Up} for each state. A multi-component fitness function provides gradient information beyond the sparse binary reward. The GA operates over 100 generations with a population of 100 individuals.

The objectives of this work are:

- Identify which combination of crossover method, mutation method, and replacement strategy produces the highest and most consistent fitness on the 8×8 map.
- Quantify the sensitivity of GA performance to mutation rate, crossover rate, population size, and tournament size.
- Determine how fitness weight configuration affects evolved behavior, comparing standard shaping against heavy hole and step penalties.
- Characterize how GA convergence changes as grid size increases from 8×8 to 16×16 and 32×32 .

Contributions of this work:

- An exhaustive comparison of all 30 operator-strategy combinations (3 crossovers $\times$ 5 mutations $\times$ 2 replacement strategies) with statistical testing on a fixed deterministic 8×8 FrozenLake-v1 map.
- Heatmap-based sensitivity analysis over mutation/crossover rates and population/tournament sizes.
- An empirical demonstration that an overly aggressive step penalty (2.0 per step) completely destabilizes GA convergence, identifying a critical fitness-shaping failure mode.
- A scalability study across 8×8 , 16×16 , and 32×32 grids revealing qualitatively distinct convergence regimes, including complete convergence failure for 32×32 under a fixed generation budget.

The remainder of this paper is organized as follows. Section II reviews related work. Section III describes the methodology. Section IV presents experimental results. Section V concludes with a discussion of findings and future directions.

II. LITERATURE REVIEW

Genetic algorithms have been established as a principled approach to reinforcement learning policy search. Whiteson [1] characterizes direct policy search—encoding a complete state-to-action mapping as an evolvable individual—as particularly suited to sparse-reward, discrete-action environments, because episode-level fitness naturally handles the credit-assignment problem that challenges value-function methods. Tournament selection is identified as providing tunable selection pressure through a single parameter, directly motivating our configurable tournament size. Faycal and Zito [2] apply this framework directly to FrozenLake-v1, evolving neural-network weights and demonstrating that two operator modifications (multi-step mutation and directed crossover) reduce convergence from over 500 to under 150 generations (Mann-Whitney U, $p < 0.01$). Though their representation is neural-network weights whereas ours is a tabular direct encoding, the core result—that crossover and mutation operator choice substantially alters convergence speed—directly motivates Experiment 1.

The systematic analysis of operator combinations and parameter interactions is the focus of two complementary works. Song et al. [3] survey reinforcement learning-assisted evolutionary algorithms (RL-EA) across five integration categories and identify adaptive operator selection as consistently outperforming fixed-operator configurations in convergence speed

and solution quality. Ma et al. [4] provide a concrete instantiation: a deep reinforcement learning agent (double DQN) selects both operator type and rates at each generation for flexible job shop scheduling, outperforming fixed-rate baselines. Their finding that adapting both operator type and rate is necessary—neither alone suffices—motivates our systematic separation of operator-type comparison (Experiment 1) from rate sensitivity (Experiments 2a and 2b).

Fitness function design is a critical dimension of GA-based navigation. Zhu et al. [5] demonstrate that potential-based reward shaping, integrated with a genetic local search framework, can increase cumulative reward by up to 39.6% in sparse-reward gridworld tasks while reducing training variance. Their theoretical result—that potential-based shaping is policy-preserving—provides formal justification for our Manhattan-distance term, which assigns non-zero fitness to every episode and acts as a surrogate potential over the goal-distance landscape. Experiment 2c directly tests the sensitivity of this shaping design.

The theoretical framework for interpreting our empirical sensitivities is provided by fitness landscape theory. Srivastava et al. [6] formalise navigability as the probability of reaching the global fitness peak via mutation and selection, showing that sign epistasis—where a mutation’s fitness effect reverses depending on genetic background—demands a careful exploration–exploitation balance. The FrozenLake policy space ($4^{64} \approx 3.4 \times 10^{38}$ possible policies) is highly epistatic: changing the action at a single state adjacent to a hole can flip fitness from maximum to minimum. This landscape structure informs the expected sensitivities in Experiments 2a, 2b, and 3, where the fitness landscape becomes progressively more rugged as grid size grows. Together, these six studies establish the theoretical and empirical foundations of our investigation, spanning GA direct policy search [1], [2], operator and parameter analysis [3], [4], fitness shaping [5], and landscape theory [6].

III. METHODOLOGY

A. Environment Setup

The environment is FrozenLake-v1 from Gymnasium, configured with `is_slippery=False` for fully deterministic transitions. The default grid size is 8×8 (64 states). A single random map is generated via `generate_random_map(size=8)` before any experiment begins and reused identically across all experimental conditions, ensuring a fair comparison across operator variants and parameter settings. The goal is at state index (7,7) (bottom-right corner). The maximum episode length is capped at $max_steps = size^2 \times 2 = 128$ steps. Evaluation is single-episode per individual; because the environment is deterministic, single-episode fitness is fully reproducible for a fixed genotype and map.

B. Individual Representation

Each individual encodes a complete deterministic tabular policy as a fixed-length integer chromosome of length 64

(one gene per state). Each gene takes a value in $\{0, 1, 2, 3\}$, corresponding to actions {Left, Down, Right, Up} respectively. Initialization is uniform random over $\{0, 1, 2, 3\}$ for all genes. This direct encoding avoids function approximation and supports a clean study of operator effects on a discrete policy space.

C. Fitness Function

The fitness of an individual is computed from a single episode as:

$$F = s \cdot w_s + \frac{w_m}{1 + d_{\text{Manhattan}}} - t \cdot w_{\text{step}} - h \cdot w_h \quad (1)$$

where $s \in \{0, 1\}$ indicates goal-reaching success; $d_{\text{Manhattan}}$ is the Manhattan distance from the agent's terminal state to the goal; t is the number of steps taken; $h \in \{0, 1\}$ indicates whether the agent fell into a hole. Default weights from `config.json`: $w_s = 100.0$, $w_m = 10.0$, $w_{\text{step}} = 0.1$, $w_h = 50.0$.

The Manhattan-distance term in (1) provides a non-zero gradient signal across all episodes regardless of goal-reaching success, acting as a surrogate potential function analogous to the potential-based shaping mechanisms of [5]. Under default weights with optimal navigation ($s = 1$, $d = 0$, $t = 14$), the theoretical maximum fitness is $F_{\text{max}} = 100 + 10 - 1.4 = 108.60$.

D. Genetic Algorithm Pipeline

The GA loop proceeds as follows:

- 1) *Initialization*: Generate $\mu = 100$ individuals with uniformly random genotypes.
- 2) *Evaluation*: Single-episode fitness evaluation for each individual.
- 3) *Selection*: Tournament selection (default tournament size $k = 3$), sampling without replacement and returning the individual with highest fitness.
- 4) *Crossover*: Applied with probability $p_c = 0.8$. Three methods are implemented: one-point (single cut-point), two-point (two cut-points exchanging a middle segment), and uniform (each gene independently drawn from either parent with equal probability). If crossover is not triggered, both parents are cloned.
- 5) *Mutation*: Applied to each offspring after crossover. Two operator categories exist: *per-gene operators* (random resetting, swap) mutate each gene independently with probability $p_m = 0.1$; *whole-individual operators* (insert, scramble, inversion) fire once per individual with probability p_m . Note that p_m has different semantics for these two categories, making direct rate comparisons between them not straightforward.
- 6) *Replacement*: Two strategies are compared. $(\mu + \lambda)$: merge parents and offspring, retain the top μ by fitness. (μ, λ) : discard parents, retain the top μ from offspring only.
- 7) *Termination*: After 100 generations.

Metrics logged per generation: best fitness, mean fitness, population success rate. Metrics reported per run: best individual's fitness, success flag, and steps taken.

E. Experimental Protocol

All experiments use $n = 5$ independent runs per condition with the same globally fixed 8×8 random map. Results are reported as means across runs. Experiments 2a–2c each vary a single axis of the default configuration while holding all other parameters at their defaults. Experiment 3 independently generates new random maps for the 16×16 and 32×32 grid sizes. Default configuration: $\mu = 100$, $T_{\text{max}} = 100$, $p_c = 0.8$, $p_m = 0.1$, $k = 3$, one-point crossover, random-resetting mutation, $(\mu + \lambda)$ strategy.

IV. EXPERIMENTAL WORK

A. Experiment 1: Operator Combination Comparison

Goal: Identify which combination of crossover operator, mutation operator, and replacement strategy achieves the highest and most consistent fitness.

Setup: All 30 combinations (3 crossovers $\times$ 5 mutations $\times$ 2 strategies) are tested for $n = 5$ independent runs each on the same fixed 8×8 map with default rates ($p_c = 0.8$, $p_m = 0.1$, $\mu = 100$, $k = 3$). Source: `results/exp1_combinations.csv`.

Findings: Five of thirty combinations achieve mean fitness 108.60 with standard deviation 0.00 across all five runs: $(\mu + \lambda)$ + uniform + random resetting; $(\mu + \lambda)$ + uniform + swap; (μ, λ) + one-point + random resetting; (μ, λ) + two-point + random resetting; and (μ, λ) + two-point + inversion. All five achieve 100% success rate and exactly 14 mean steps, matching the theoretical fitness maximum. The zero standard deviation reflects that the GA converged to the same optimal policy on every run—a consequence of the fully deterministic environment. The t -test between the top two configurations produced $p = \text{NaN}$, because the Welch t -test is undefined when both groups have zero variance. This is not a test failure; it reflects complete convergence: any run of these configurations on this deterministic task finds an optimal 14-step policy every time. The worst result was $(\mu + \lambda)$ + one-point + inversion (mean = -7.81 , success rate = 0%).

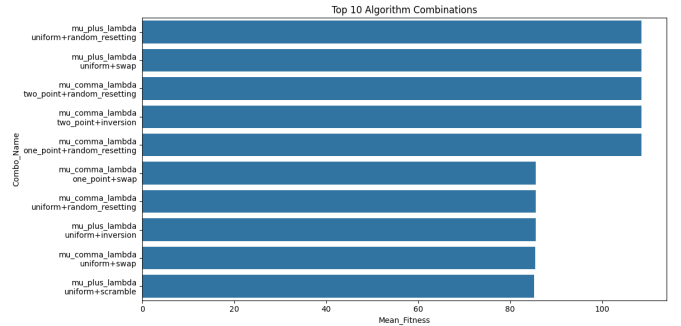


Fig. 1. Top 10 operator combinations ranked by mean fitness over 5 runs. Multiple combinations tie at the maximum fitness of 108.60; configurations using permutation-style mutation operators cluster at the bottom.

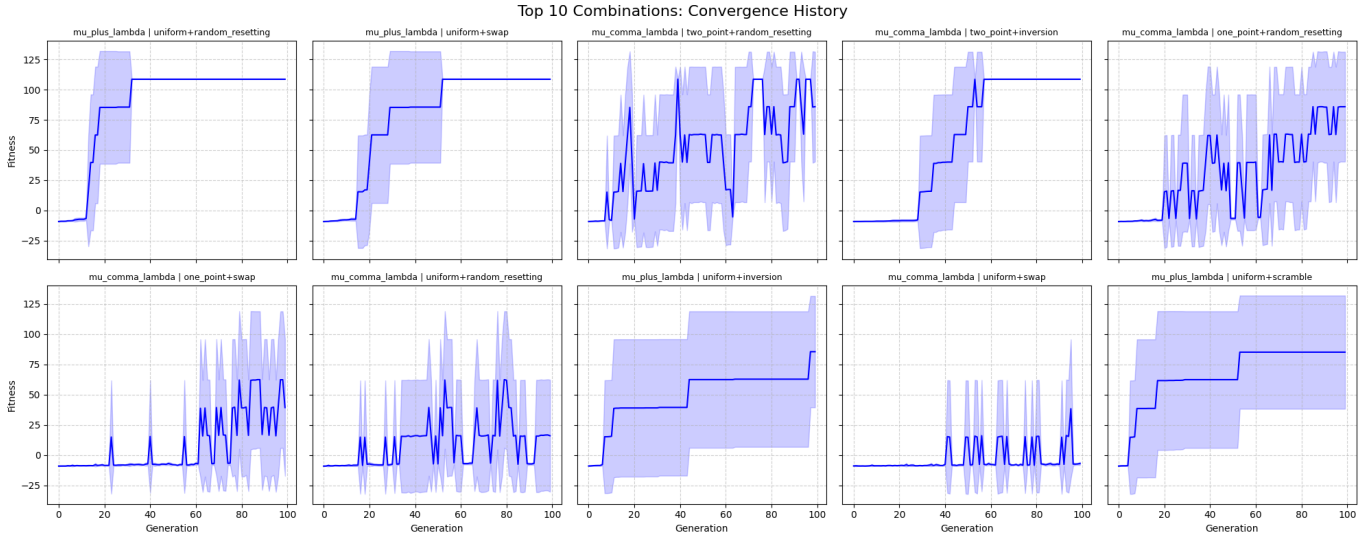


Fig. 2. Convergence curves (mean $\pm$ std over 5 runs) for the top 10 operator combinations. Configurations achieving maximum fitness converge sharply within approximately 50 generations; lower-ranked configurations exhibit slower ascent and higher variance.

TABLE I
SELECTED OPERATOR COMBINATIONS FROM EXPERIMENT 1 ($n = 5$ RUNS EACH). TOP FIVE AND THREE WORST CONFIGURATIONS ARE SHOWN.

Strat.	CX	Mut.	Mean F	Std	SR
$\mu + \lambda$	uniform	rnd. reset	108.60	0.00	1.00
$\mu + \lambda$	uniform	swap	108.60	0.00	1.00
μ, λ	one-point	rnd. reset	108.60	0.00	1.00
μ, λ	two-point	rnd. reset	108.60	0.00	1.00
μ, λ	two-point	inversion	108.60	0.00	1.00
$\mu + \lambda$	one-point	insert	38.88	56.94	0.40
μ, λ	one-point	insert	-7.71	1.43	0.00
$\mu + \lambda$	one-point	inversion	-7.81	0.93	0.00

Strat.=strategy; CX=crossover; Mut.=mutation; SR=success rate.

Interpretation: Uniform crossover preserves diversity by independently recombining each gene, which is well-suited to a tabular representation where gene position carries no intrinsic meaning. Random resetting directly explores the discrete action space by replacing a gene with a uniformly sampled value, making it the most natural mutation operator for this encoding. In contrast, insert, scramble, and inversion are permutation-style operators that rearrange gene order: they preserve the multiset of actions in the genotype but reassign which action maps to which state. For tabular direct encoding, rearranging assignments is generally harmful because each gene’s fitness contribution depends on its state-index semantics, not its position relative to neighboring genes [2].

Caveats: $n = 5$ runs is modest; zero-variance results reflect the deterministic task structure rather than universal robustness across map topologies.

B. Experiment 2a: Mutation Rate and Crossover Rate Sensitivity

Goal: Map how fitness varies jointly with mutation rate and crossover rate.

Setup: Grid search over $p_m \in \{0.01, 0.05, 0.2\}$ and $p_c \in \{0.5, 0.7, 0.9\}$ (9 conditions, $n = 5$ runs each). Default operator: one-point crossover, random-resetting mutation, $(\mu + \lambda)$ strategy. Source: results/exp2a_heatmap_data.csv.

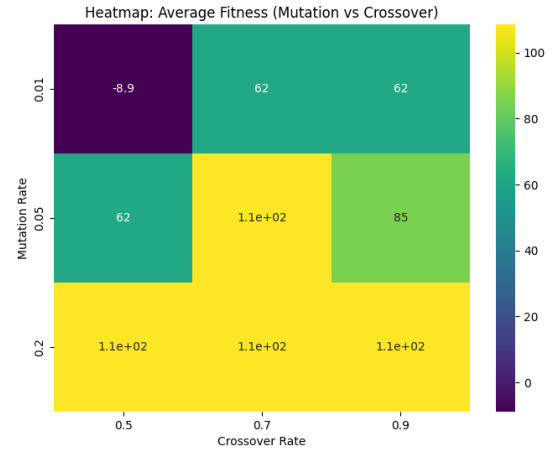


Fig. 3. Average fitness as a function of mutation rate (rows) and crossover rate (columns). Low mutation ($p_m = 0.01$) with low crossover ($p_c = 0.5$) yields catastrophic failure (-8.88); mutation rate ≥ 0.2 achieves peak fitness across all crossover rates.

The heatmap in Fig. 3 reveals a clear pattern. At $p_m = 0.01$, $p_c = 0.5$, the GA fails entirely (mean $= -8.88$, 0% success): minimal mutation combined with infrequent crossover means that roughly half of offspring per generation are exact clones of their parents, with the other half receiving only marginal perturbations. Population diversity collapses quickly, trapping the search in a low-fitness local optimum. Increasing p_c to 0.7–0.9 at the same $p_m = 0.01$ recovers partial performance (mean ≈ 61.6), indicating that crossover can partially compensate for

low mutation. At $p_m = 0.2$, peak fitness (108.60) is achieved across all three crossover rates, suggesting that sufficient per-gene diversity maintenance from mutation makes crossover rate secondary.

Interpretation: This pattern is consistent with fitness landscape theory [6]: the FrozenLake policy space contains sign epistasis where single-gene changes at critical states can drastically alter fitness. High mutation rates maintain the diversity required to escape such local optima; crossover rate matters most when mutation rate is insufficient on its own.

Caveats: Coarse three-level grid; results are specific to one-point crossover and random-resetting mutation.

C. Experiment 2b: Population Size and Tournament Size Sensitivity

Goal: Quantify the joint effect of population size and tournament size (selection pressure) on GA performance.

Setup: Grid search over $\mu \in \{20, 50, 200\}$ and $k \in \{2, 5, 10\}$ (conditions with $k > \mu$ are omitted), $n = 5$ runs each. Source: results/exp2b_sensitivity_data.csv.

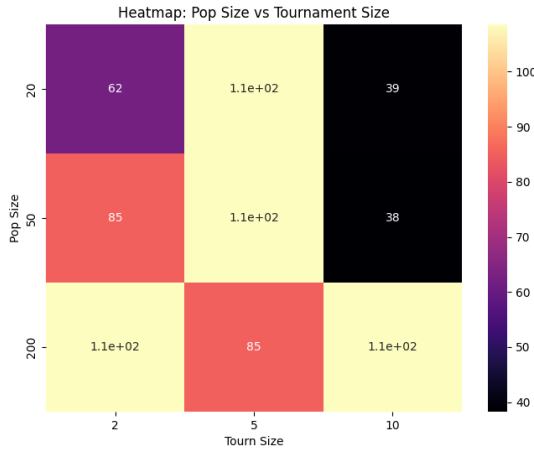


Fig. 4. Average fitness as a function of population size (rows) and tournament size (columns). Tournament size $k = 5$ is optimal for small populations; excessive selection pressure ($k = 10$) causes premature convergence at small population sizes.

As shown in Fig. 4, tournament size $k = 5$ with $\mu \in \{20, 50\}$ achieves peak fitness (108.60). Tournament size $k = 10$ with these small populations yields mean fitness ≈ 38 , reflecting excessive selection pressure: a tournament of 10 from a population of 20 amounts to selecting from the top half at each step, rapidly concentrating the population around a few high-fitness individuals and eliminating diversity needed to escape local optima. Population $\mu = 200$ is tolerant of both $k = 2$ and $k = 10$ (both achieve 108.60), likely because 200 individuals provide sufficient diversity to buffer against extreme selection pressures. The anomalous result at $\mu = 200$, $k = 5$ (85.17 instead of 108.60) likely reflects sampling variance given $n = 5$.

Interpretation: Selection pressure must be matched to population size. The navigability analysis in [6] predicts that loss of diversity through excessive selection is particularly harmful in rugged, epistatic landscapes—exactly the profile of the FrozenLake policy space.

Caveats: Only 5 runs per condition; the $\mu = 200$, $k = 5$ anomaly warrants further investigation.

D. Experiment 2c: Fitness Weight Sensitivity

Goal: Determine how the choice of fitness weight configuration alters evolved behavior and final performance.

Setup: Three weight configurations tested, $n = 5$ runs each. *Standard:* $w_s = 100$, $w_m = 10$, $w_{\text{step}} = 0.1$, $w_h = 50$. *Heavy_Hole:* w_h increased to 200.0. *Heavy_Step:* w_{step} increased to 2.0. Source: results/exp2c_weights.csv.

TABLE II
FITNESS WEIGHT SENSITIVITY: EXPERIMENT 2C RESULTS ($n = 5$ RUNS EACH)

Weight Config	Avg Fitness	Succ. Rate	Avg Steps
Standard	108.60	1.00	14.0
Heavy_Hole	85.17	0.80	31.2
Heavy_Step	-51.29	0.00	1.0

Table II and Fig. 5 show the results. The standard configuration achieves perfect fitness (108.60), 100% success, and 14 mean steps—the minimum path length from (0,0) to (7,7). The Heavy_Hole configuration ($w_h = 200$) reduces success to 80% with 31.2 mean steps, suggesting the GA evolves overly cautious policies that take longer, circuitous routes to avoid any proximity to holes, at the cost of overall success reliability.

The Heavy_Step configuration ($w_{\text{step}} = 2.0$) is a catastrophic failure: 0% success, 1.0 mean steps, and mean fitness -51.29 . A step penalty of 2.0 per step would incur -256 over 128 steps, completely overwhelming the positive signals. The GA learns to minimise steps by immediately terminating—the easiest path is to fall into the first adjacent hole in a single step. The observed mean fitness of -51.29 is consistent with the equation in (1): a 1-step terminal transition into a hole at state (1,0) (Manhattan distance 13 from the goal) gives $0 + 10/14 - 2.0 - 50.0 \approx -51.3$.

Interpretation: This is a concrete illustration of the fitness shaping pitfall noted by [5]: a penalty that dominates the fitness function causes the GA to optimise for that component alone at the expense of the true objective. The standard balanced configuration is essential for reliable convergence.

Caveats: Only three weight configurations tested; intermediate values of w_{step} between 0.1 and 2.0 were not explored.

E. Experiment 3: Scalability Across Grid Sizes

Goal: Characterize how GA convergence rate and final performance change as grid size (and thus policy-space size) grows.

Setup: Three grid sizes— 8×8 (genotype length 64), 16×16 (length 256), 32×32 (length 1024)—tested with $\mu = 150$, $T_{\text{max}} = 150$, $n = 5$ runs each. The 8×8 map uses

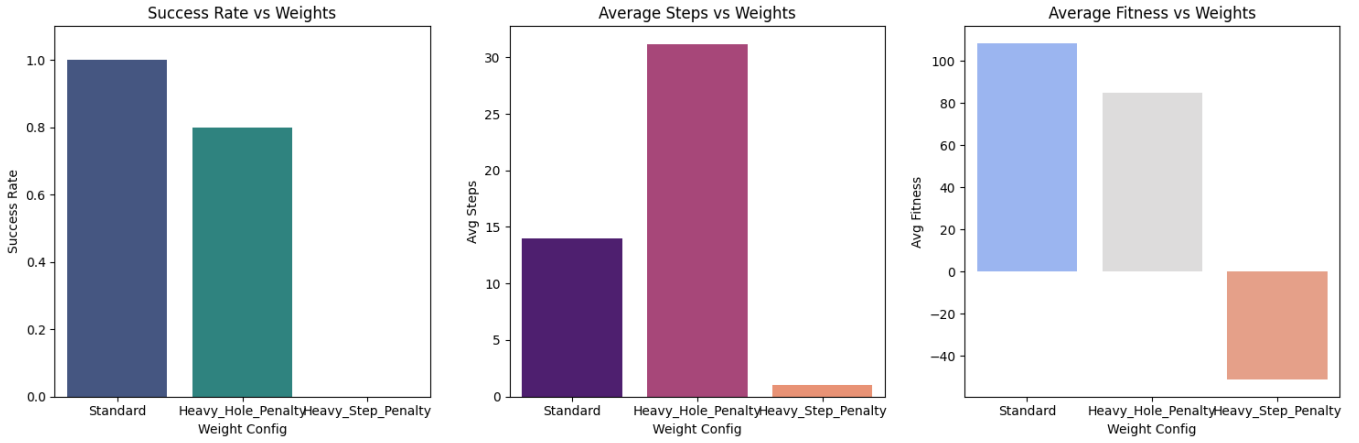


Fig. 5. Effect of fitness weight configuration on success rate (left), average steps (center), and average fitness (right). The Heavy Step Penalty collapses performance to 0% success and 1 mean step; the Standard configuration achieves perfect success at minimum path length.

the same global map as previous experiments; 16×16 and 32×32 use independently generated random maps. Source: results/exp3_scalability_ci.png.

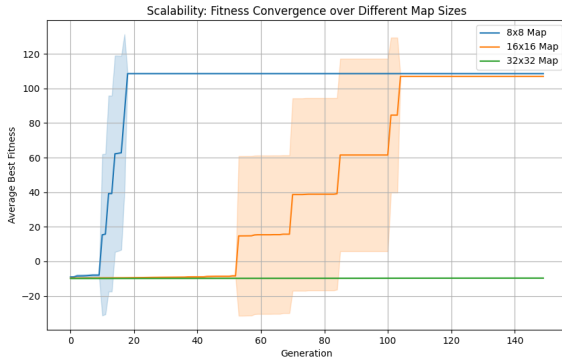


Fig. 6. Scalability study: mean best fitness with shaded variability band across 150 generations for grid sizes 8×8 , 16×16 , and 32×32 ($\mu = 150$, $n = 5$ runs). The 8×8 curve reaches a high-fitness plateau by approximately generation 20; the 16×16 curve improves later via step-like jumps, reaching a comparable plateau only after generation 100; the 32×32 curve remains near its initial fitness level throughout, indicating failure to discover higher-quality policies under the same budget.

As shown in Fig. 6, the 8×8 and 16×16 conditions both achieve high final fitness, but their convergence dynamics differ substantially. The 8×8 curve stays near its initial level for the first ≈ 10 generations, then rises sharply and reaches a plateau around ≈ 108 by approximately generation 20, remaining essentially flat thereafter. In contrast, the 16×16 curve stays near baseline for much longer, beginning to improve only around generation 50 and rising in a series of step-like jumps—roughly in the generation ranges 50–70, 80–90, and 100–105—before reaching a comparable plateau near ≈ 108 shortly after generation 100.

The 32×32 curve, however, remains almost flat around its initial fitness level (approximately -10) across all 150 generations, with no visible sustained improvement. This indicates

that for the 32×32 random map instance used here, the fixed search budget (150 individuals $\times$ 150 generations) and the default operator/parameter configuration were insufficient to reliably discover higher-quality policies.

Interpretation: With increasing genotype length ($64 \rightarrow 256 \rightarrow 1024$), the effective search difficulty increases sharply. This manifests in the plot as (i) much later convergence for 16×16 relative to 8×8 , and (ii) a complete lack of progress for 32×32 under the same budget. This qualitative pattern is consistent with the fitness landscape perspective of [6]: as dimensionality grows, beneficial mutation pathways become rarer and the landscape harder to navigate, making convergence increasingly dependent on larger evaluation budgets, better-tuned operator rates, and/or adaptive operator/parameter control [3], [4].

Caveats: The 16×16 and 32×32 conditions use independently generated random maps, so grid-size effects are confounded with map-instance difficulty; the flat 32×32 curve may reflect an unusually difficult map instance rather than grid size alone. Additionally, the shaded regions reflect variability across only 5 runs and should be interpreted as indicative rather than as definitive confidence intervals. A stronger scalability conclusion would require multiple random map instances per size and a generation budget scaled with genotype length.

V. CONCLUSION AND DISCUSSION

Key Empirical Findings:

- Five of thirty tested operator combinations achieve the theoretical maximum fitness (108.60, 100% success, 14 mean steps) with zero variance across five independent runs. Per-gene mutation operators (random resetting, swap) dominate successful configurations; whole-individual positional operators (insert, scramble, inversion) are generally poorly suited to the tabular direct encoding.

- Operator semantics matter independently of rate: insert, scramble, and inversion rearrange gene positions rather than replacing gene values, making them structurally mismatched to a representation where gene identity—not position—carries all policy information.
- Mutation rate is the primary diversity-maintenance lever. A rate of 0.2 achieves peak fitness across all tested crossover rates; at 0.01, fitness is critically sensitive to crossover rate and can collapse to -8.88 .
- Tournament size must be matched to population size. Tournament size 10 with populations of 20–50 causes premature convergence (mean fitness ≈ 38); tournament size 5 is optimal for these smaller populations.
- Fitness shaping balance is as critical as operator design. An aggressive step penalty (2.0 per step) entirely destroys convergence, reducing success to 0% and evolving policies that immediately self-terminate.
- The scalability study reveals qualitatively distinct convergence regimes: 8×8 converges rapidly by generation 20, 16×16 converges via step-like jumps reaching a comparable high-fitness plateau only after generation 100, and 32×32 shows no meaningful improvement within the same 150-generation budget, demonstrating that search difficulty grows sharply with genotype length.

Limitations:

- $n = 5$ runs per condition is modest; higher-variance configurations may require 20–30 runs for reliable statistical conclusions.
- All experiments share one fixed random 8×8 map; results may not generalise to other map topologies.
- The mutation rate parameter has different semantic interpretations for per-gene and whole-individual operators, confounding direct comparisons between these operator classes.
- Experiment 3 uses different maps for 16×16 and 32×32 , so grid-size effects and map-instance difficulty are confounded; the flat 32×32 curve may reflect an unusually difficult map instance rather than grid size alone. Additionally, the shaded variability bands reflect only 5 runs and are indicative rather than definitive confidence intervals.

Future Work:

- Replace fixed-rate operator selection with an adaptive RL-based scheme trained on the empirical baselines established in Experiments 1–2, following the RL-EA framework of [3], [4].
- Extend Experiment 3 to test multiple random map instances per grid size, separating topology effects from scaling effects.
- Normalise mutation magnitude across operator types (e.g., fix the expected number of changed genes) to enable fair comparisons between per-gene and whole-individual operators.
- Investigate multi-episode evaluation with a stochastic map (`is_slippery=True`) to test policy robustness

under uncertainty.

- Explore intermediate fitness weight values (especially w_{step} between 0.1 and 2.0) to identify the threshold at which the step penalty becomes counterproductive.

AI USAGE STATEMENT

Generative AI tools (Gemini, Claude) were used as assistive supports throughout the project, without delegating full implementation to the model. Specifically, AI was used (i) during the initial phase to review selected references and facilitate brainstorming about problem framing, fitness shaping, and experiment design; (ii) as a consultant to discuss hyperparameter choices (e.g., population size, mutation/crossover rates, tournament size) and replacement strategies; (iii) for troubleshooting and debugging when the team encountered uncertain implementation issues (e.g., operator semantics, edge cases, or unexpected experimental behavior); and (iv) to help refine the presentation of plots and tables and to edit and restructure the report text, including converting the finalized content into IEEE-compliant L^AT_EX.

All AI suggestions and drafted text were critically reviewed, discussed, and verified by the group. Every numerical claim was cross-checked against the source code and the generated files under `results/`. The final methodological decisions, code, experiments, and interpretations are the authors’ own.

DATA AVAILABILITY

The source code, configuration file (`config.json`), and experimental results that support the findings of this paper are included in the project package. All result plots (`.png`) and numerical summaries (`.csv`) are located in the `results/` directory. The configuration file specifies all hyperparameters exactly as used in the experiments.

MEMBER CONTRIBUTIONS

Arda Çağan Keser was responsible for project conceptualization and problem formulation; implementation of the environment interface and the multi-component fitness function; development of the full GA pipeline; design and implementation of all five experiments; execution of all experimental runs; and analysis and interpretation of results. He also authored the Methodology and Experimental Work sections of this report.

Osman Oğuz Erol was responsible for the implementation of all three crossover operators (one-point, two-point, and uniform crossover); development of the visualization and result-plotting scripts; hyperparameter configuration; and analysis and interpretation of results. He also authored the Abstract and Introduction sections of this report.

Utkan Keser was responsible for the implementation of all five mutation operators (random resetting, swap, insert, scramble, and inversion); design and execution of the statistical test (Welch t -test); literature search and citation management; and analysis and interpretation of results. He also authored the Literature Review and Conclusion and Discussion sections of this report.

Shahram Saraeipour contributed to initial topic selection and problem scoping discussions, performed spot-checks on selected experimental outputs, assisted with debugging during implementation, and reviewed the final L^AT_EX report formatting.

REFERENCES

- [1] S. Whiteson, “Evolutionary computation for reinforcement learning,” in *Reinforcement Learning: State-of-the-Art*, Springer, 2012, pp. 325–355.
- [2] T. Faycal and C. Zito, “Direct mutation and crossover in genetic algorithms applied to reinforcement learning tasks,” *arXiv preprint arXiv:2201.04815*, 2022.
- [3] Y. Song, Y. Wu, Y. Guo, R. Yan, P. N. Suganthan, Y. Zhang, W. Pedrycz, S. Das, R. Mallipeddi, O. S. Ajani, and Q. Feng, “Reinforcement learning-assisted evolutionary algorithm: A survey and research opportunities,” *Swarm Evol. Comput.*, vol. 86, p. 101517, 2024.
- [4] J. Ma, W. Gao, and W. Tong, “A deep reinforcement learning assisted adaptive genetic algorithm for flexible job shop scheduling,” *Eng. Appl. Artif. Intell.*, vol. 149, p. 110447, 2025.
- [5] Z. Zhu, Z. Chen, C. Zhu, W. Si, and F. Wang, “Optimizing potential-based reward automata in partially observable reinforcement learning using genetic local search,” *Eng. Appl. Artif. Intell.*, vol. 169, p. 114054, 2026.
- [6] M. Srivastava, C. Bank, J. Krug, and S. G. Das, “Evolution as fitness landscape navigation: Concepts, measures, and emerging questions,” *arXiv preprint arXiv:2604.17036*, 2026.